Android Compose UI

Interview Design Specifications

12 exercises · UX mockups + annotations + Compose keys

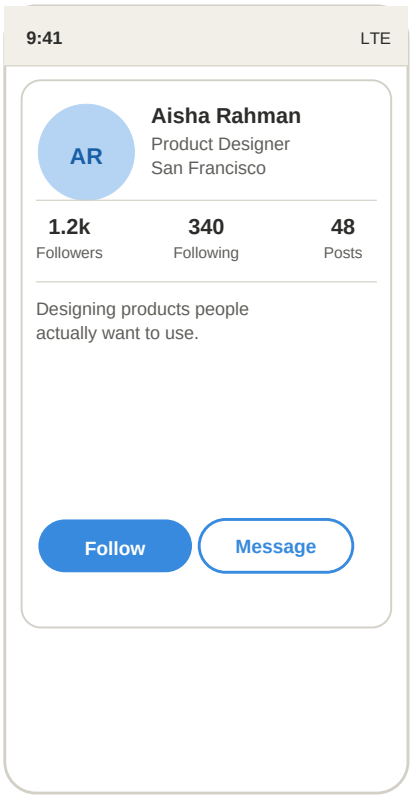
Study each screen — identify containers, state, composable boundaries

How to use this PDF: Each exercise shows a realistic Android UI mockup as an interviewer might present it. Study the screen, identify the composable tree, then check the annotations. Practice implementing each from memory in CoderPad.

UX design exercises

1 Profile card

Layout | Warm-up



UI components:

- Avatar circle (initials + background color)
- Name + subtitle Column
- Follow (filled) + Message (outlined) buttons

Key annotations:

Avatar

```
Box + Modifier.clip(CircleShape) + background
```

Stats Row

```
Row, each col uses Modifier.weight(1f)
```

Buttons

```
Button + OutlinedButton in Row with Spacer
```

Core Compose:

Row/Column nesting · Modifier.clip(CircleShape) · OutlinedButton

State:

Stateless — accept User data class as parameter

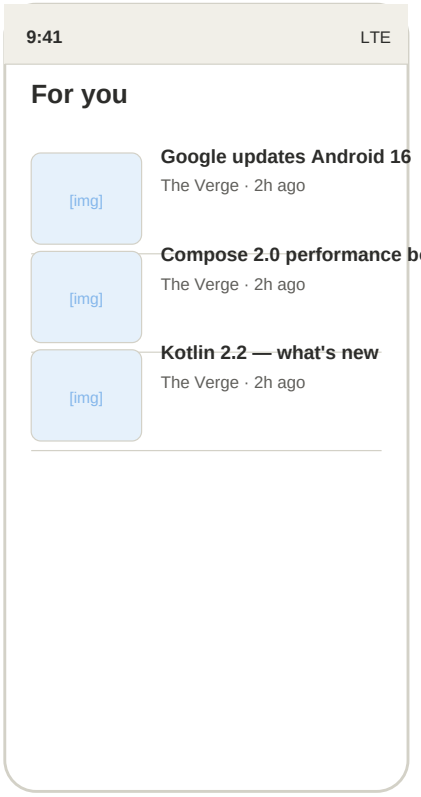
Clarify first:

→ Avatar from URL (Coil) or initials only?

→ Buttons functional or visual-only?

02 Feed / news list

LazyList | Core



UI components:

- Toolbar with title
- Thumbnail placeholder (Box)
- Article title — 2 line max
- Source + timestamp row

Key annotations:

LazyColumn

```
items(articles, key = { it.id }) — always add key
```

Image

```
Box with clip + background simulates AsyncImage
```

Overflow

```
maxLines = 2, overflow = TextOverflow.Ellipsis
```

Core Compose:

LazyColumn · items(key=) · TextOverflow.Ellipsis

State:

Stateless — list passed from ViewModel

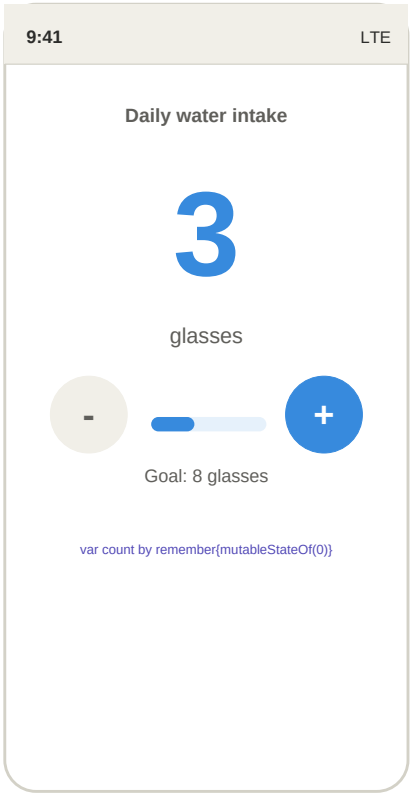
Clarify first:

→ How many items? (Column vs LazyColumn)

→ Does tapping navigate to detail?

03 Counter widget

State | Warm-up



UI components:

- Large numeric display
- + increment button
- – decrement button (disabled at 0)
- Progress bar toward goal

Key annotations:

State

```
var count by remember { mutableStateOf(0) }
```

Disabled

```
Button(enabled = count > 0) auto 38% alpha
```

Progress

```
LinearProgressIndicator(progress = count/goal.toFloat())
```

Core Compose:

remember · mutableStateOf · by delegate · Button(enabled=)

State:

var count — local; hoist to ViewModel for persistence

Clarify first:

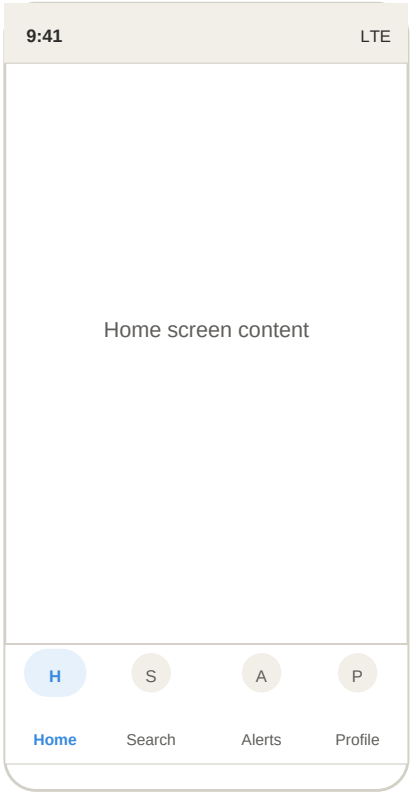
→ *Is there a max value too?*

→ *Does count persist across navigation?*

---

04 Bottom navigation bar

Compose | Core



UI components:

- Scaffold wrapping the screen
- NavigationBar at the bottom
- 4 NavigationBarItems
- Selected tab: tinted icon + pill background

Key annotations:

Scaffold

```
Scaffold(bottomBar = { NavigationBar { ... } })
```

Item

```
NavigationBarItem(selected, onClick, icon, label)
```

State

```
var selectedIndex by remember { mutableStateOf(0) }
```

Core Compose:

Scaffold · NavigationBar · NavigationBarItem · selectedIndex

State:

var selectedIndex: Int — drives active tab

Clarify first:

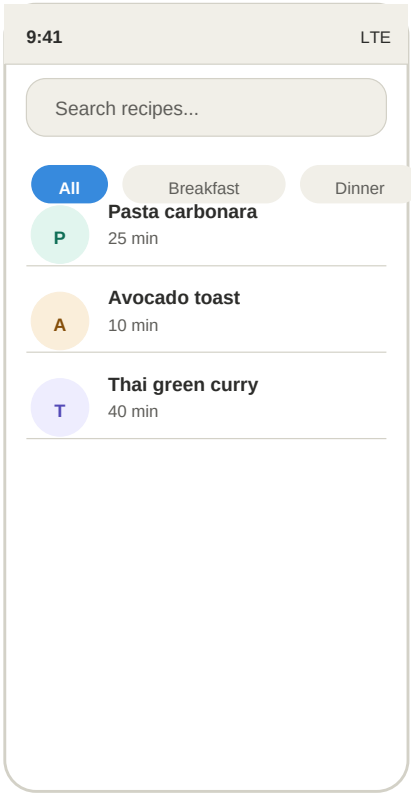
→ *Wired to NavGraph or visual-only?*

→ *Unread badge on Alerts?*

05

Search + filter chips

State | Core



UI components:

- TextField search bar
- LazyRow of FilterChips
- Reactive filtered LazyColumn
- Empty state message

Key annotations:

TextField

```
value = query, onValueChange = { query = it }
```

Chips

```
LazyRow { items { FilterChip(selected, onClick) } }
```

Filter

```
derivedStateOf { list.filter { query & chip match } }
```

Core Compose:

TextField · LazyRow · FilterChip · derivedStateOf

State:

query: String + selectedChip: String → feed derivedStateOf

Clarify first:

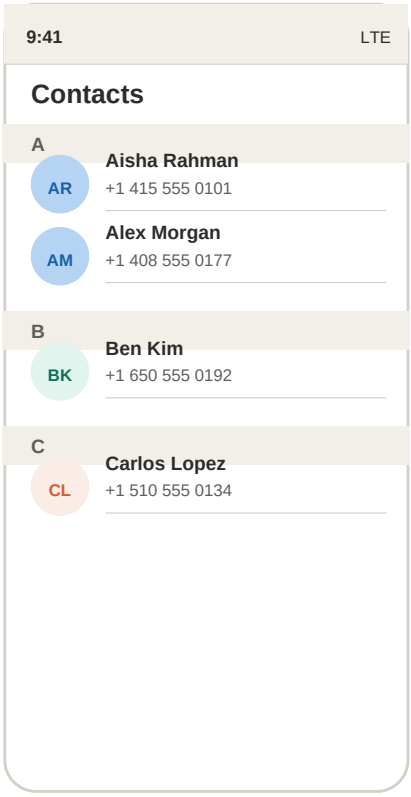
→ *Single-select or multi-select chips?*

→ *Empty state view needed?*

06

Grouped contact list

LazyList | Advanced



UI components:

- Toolbar
- Sticky alphabetical section headers
- Contact rows with avatar circles
- Phone number subtitle

Key annotations:

Group

```
contacts.groupBy { it.name.first().uppercaseChar() }
```

Sticky

```
stickyHeader(key = letter) { SectionHeader(letter) }
```

Items

```
items(group, key = { it.id }) { ContactRow(it) }
```

Core Compose:

LazyColumn · stickyHeader · groupBy · key param

State:

Stateless — sorted + grouped contacts from ViewModel

Clarify first:

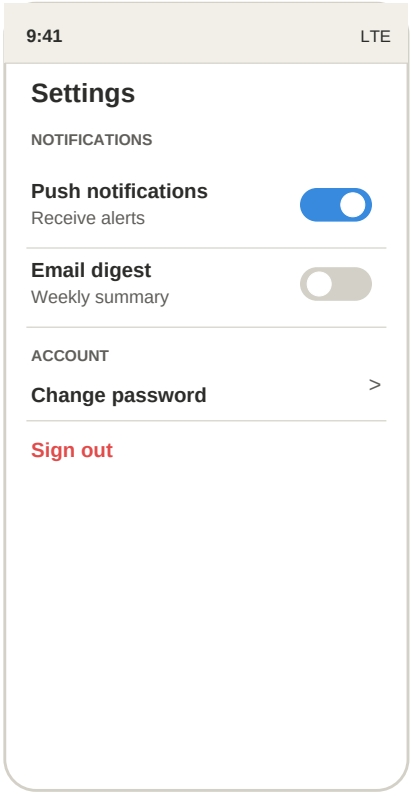
→ *Alpha index rail on right edge?*

→ *Tap = navigate or call?*

---

9 Settings screen

Layout | Core



UI components:

- Section labels (UPPERCASE)
- Toggle rows with Switch
- Navigation rows (chevron)
- Destructive action (red text)

Key annotations:

ListItem

```
ListItem(headlineContent, trailingContent = { Switch })
```

Switch

```
Switch(checked = state, onCheckedChange = { state = it })
```

Destructive

```
color = MaterialTheme.colorScheme.error
```

Core Compose:

Column · ListItem · Switch · clickable · error color

State:

var pushEnabled, var emailEnabled — hoist to ViewModel

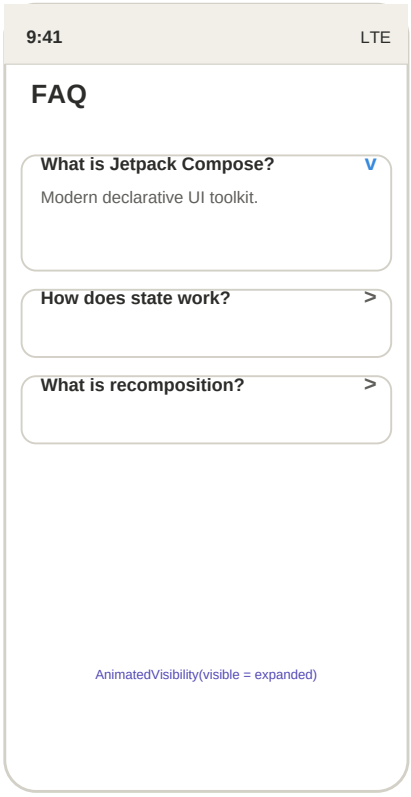
Clarify first:

→ *Scrollable or fixed?*

→ *Connected to real DataStore?*

08 Expandable accordion

Compose | Core



UI components:

- Clickable card header row
- Chevron rotates 180° when expanded
- AnimatedVisibility body text
- Multiple independent items

Key annotations:

State

```
var expanded by remember { mutableStateOf(false) }
```

Visibility

```
AnimatedVisibility(visible = expanded) { Text(...) }
```

Rotation

```
animateFloatAsState(if(expanded) 180f else 0f)
```

Core Compose:

AnimatedVisibility · animateFloatAsState · rotate() modifier

State:

Each AccordionItem owns local expanded state

Clarify first:

→ *One card open at a time (shared state) or independent?*

→ *Animation required?*

09 Multi-step form

State | Advanced

UI components:

- Step indicator (circles + connecting lines)
- Conditional step content
- Back / Continue buttons
- Validate before advancing

Key annotations:

Step

```
var currentStep by remember { mutableStateOf(1) }
```

Content

```
AnimatedContent(targetState = currentStep) { when(it){...} }
```

Buttons

```
Back hidden on step 1; Submit label on last step
```

Core Compose:

AnimatedContent · when() switch · Scaffold bottomBar for CTA

State:

var currentStep: Int + each step's form field states

Clarify first:

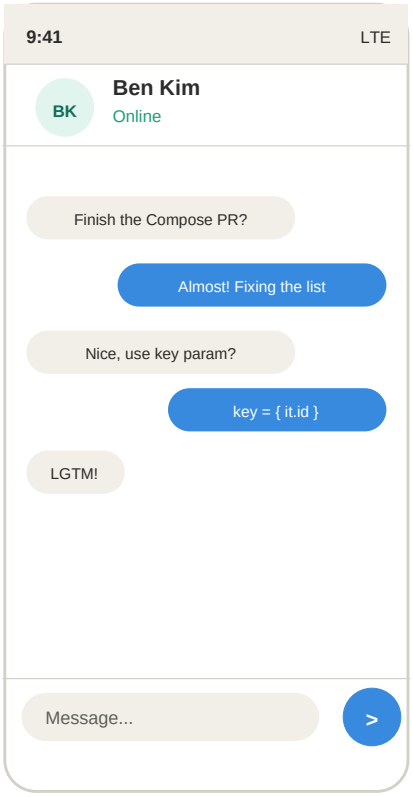
→ *Back on step 1 = dismiss?*

→ *Validate before Next?*

---

10 Chat message list

LazyList | Advanced



UI components:

- Contact header bar
- Reversed LazyColumn (newest at bottom)
- Sent bubbles (right, blue)
- Received bubbles (left, gray)
- Input bar + send button

Key annotations:

Reverse

```
LazyColumn(reverseLayout = true)
```

Alignment

```
Arrangement.End (sent) / .Start (received)
```

Bubble

```
RoundedCornerShape with one flat corner per side
```

Core Compose:

LazyColumn(reverseLayout) · Row Arrangement · clip(RoundedCornerShape)

State:

var inputText: String — message list from ViewModel StateFlow

Clarify first:

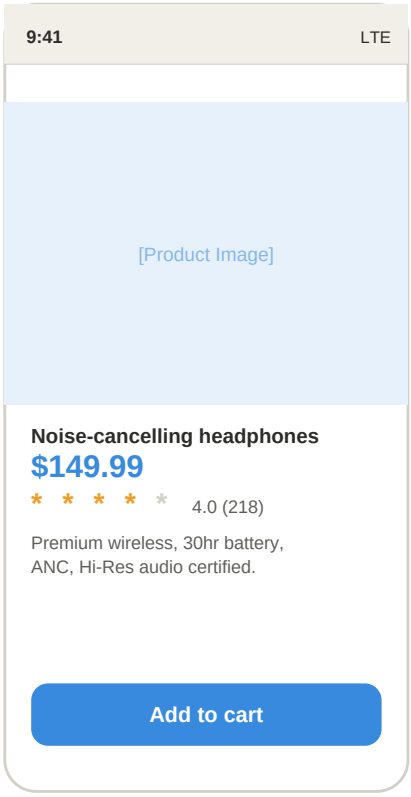
→ *Auto-scroll on new message?*

→ *Timestamps grouped or per-message?*

---

1 Product detail page

Layout | Core



UI components:

- Full-width hero image
- Name, price, star rating row
- Scrollable description
- Add to cart CTA pinned at bottom

Key annotations:

Scroll

```
Modifier.verticalScroll(rememberScrollState())
```

Image

```
contentScale = ContentScale.Crop fills width
```

CTA

```
Scaffold(bottomBar = { Button }) keeps CTA visible
```

Core Compose:

verticalScroll · ContentScale.Crop · Scaffold bottomBar

State:

Stateless display; pass addToCart lambda in

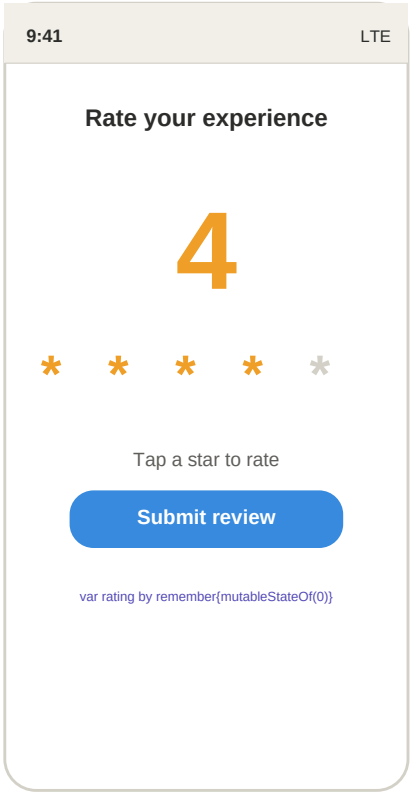
Clarify first:

→ *CTA always visible (Scaffold) or scrolls?*

→ *Multiple images / pager?*

1/2 Rating bar

Compose | Warm-up



UI components:

- Large numeric display
- Row of 5 tappable star icons
- Filled (amber) vs outline (gray) by index
- Submit review button

Key annotations:

State

```
var rating by remember { mutableStateOf(0) }
```

Icons

```
repeat(5){ i -> if(i < rating) Star else StarBorder }
```

Tap

```
Modifier.clickable { onRatingChange(index + 1) }
```

Core Compose:

repeat() · Icons.Default.Star/StarBorder · clickable

State:

var rating: Int — or stateless with onRatingChange callback

Clarify first:

→ *Half-star or whole numbers only?*

→ *Read-only or interactive?*

Exercise index

#	Screen	Tag	Difficulty	Primary Compose concept
01	Profile card	Layout	Warm-up	Row/Column nesting
02	Feed / news list	LazyList	Core	LazyColumn
03	Counter widget	State	Warm-up	remember
04	Bottom navigation bar	Compose	Core	Scaffold
05	Search + filter chips	State	Core	TextField
06	Grouped contact list	LazyList	Advanced	LazyColumn
07	Settings screen	Layout	Core	Column
08	Expandable accordion	Compose	Core	AnimatedVisibility
09	Multi-step form	State	Advanced	AnimatedContent
10	Chat message list	LazyList	Advanced	LazyColumn(reverseLayout)
11	Product detail page	Layout	Core	verticalScroll
12	Rating bar	Compose	Warm-up	repeat()